

UČNI SCENARIJ

01 PREDSTAVITEV

Naslov učnega scenarija	Korak po koraku, algoritmi vsakdanjika.
Tema (glede na področje RIN)	1. Računalniški sistemi 2. Podatki in analiza 3. Algoritmi in programiranje 4. Omrežja in internet 5. Učinki računalništva in informatike
Povzetek učnega scenarija	V okviru treh sklopov učnih aktivnosti učenci spoznajo osnove algoritmov, programiranja ter uporabo spremenljivk, funkcij in zank pri reševanju problemov. Z uporabo spletnega orodja Code.org spoznajo vizualno in tekstovno programiranje v jeziku Java, ter usvojijo osnovne koncepte spremenljivk, funkcij in blokovnega programiranja ter pomen modularnosti. Z aplikacijo Lightbot spoznajo pomen pravilnega zaporedja ukazov, funkcij, pri tem pa razvijajo logično mišljenje, natančnost in vztrajnost pri doseganju cilja. Z uporabo spletnega orodja Pišek pa nadgradijo razumevanje algoritmov, saj s pomočjo »želvje kode« spoznajo uporabo zaporednih ukazov, preprostih in gnezdenih zank, prepoznavanje vzorcev, odpravljanje napak ter iskanje učinkovitejših programskih rešitev. Sklopi učnih aktivnosti tako učencem omogočajo postopno razvijanje sistematičnega reševanja problemov, preverjanja pravilnosti rešitev in algoritmičnega mišljenja.
Ključne besede	Algoritmi, ukazi, navodila, programi
Licenca dostopnosti in uporabe učnega scenarija	 CC BY-SA
Avtor(ji) učnega scenarija na VIZ	VIZ: OŠ Vojke Šmuc Izola • Burgar Mojca • Bedrač Dejan • Novak Gorazd • Udovič Marko

02 KONTEKST IZVEDBE IN PRIPRAVA

Izobraževalna stopnja, obdobje učencev	7. - 9. razred
Predznanje	• Osnovno razumevanje algoritma (vedo, da je algoritem zaporedje natančnih korakov za rešitev problema). • Osnovna digitalna pismenost (zanesljivo uporabljajo računalnik, miško in brskalnik). • Poznavanje koncepta napake v algoritmu . • Sposobnost branja preprostih navodil (znajo razbrati problem, določiti cilj in slediti zapisanemu opisu naloge v orodju). • Osnovna prostorska predstava (znajo slediti premikom "naprej", "levo", "desno", "obrnj se", kar je ključno za željo kodo).
Trajanje izvedbe	Tri pedagoške ure.
Viri za oblikovanje priprave	• https://code.org/en-US • https://www.gameflare.com/online-game/light-bot/ • https://pisek.acm.si/contents/4907-4902/

03 NAMEN IN UČNI CILJI (OPERATIVNI)

Namen	Namen učnih aktivnosti je pri učencih razvijati razumevanje osnovnih konceptov algoritmov in programiranja ter strukturiranega reševanja problemov. Učenci spoznajo, da je program sestavljen iz natančnega niza ukazov, pri čemer je pravilno zaporedje ključno za doseg cilja. V okviru aktivnosti učenci načrtujejo preproste algoritme, uporabljajo spremenljivke, funkcije, različne vrste zank in pogojne stavke ter prepoznavajo ponavljajoče se vzorce. Ob tem se učijo razčleniti problem, oblikovati ustrezen algoritem, preverjati pravilnost rešitev, odkrivati in odpravljati napake ter izboljševati ustvarjene algoritme. Aktivnosti hkrati spodbujajo samostojno raziskovanje, prilagajanje obstoječih rešitev novim problemom, utemeljevanje izbranih rešitev, vztrajnost in samostojnost.
Učni cilji (operativni)	• Učenec razume, da se stvarni problem za potrebe reševanja v računalniško informacijskem svetu modelira v računalniško informacijski problem, za rešitev katerega načrtujemo algoritem. • Učenec prepozna in opiše zaporedja ukazov kot vzorec. • Učenec zna zapisati ponavljanje vzorca z uporabo zanke. • Učenec zna napovedati, predvidevati, oceniti rezultat izvajanja zanke. • Učenci na osnovi izbora in kombinacije krmilnih konstruktov znajo doseči kompleksnejše delovanje programa.

	• Učenec spremlja izvajanje programa in preverja, ali program deluje v skladu s pričakovanji. • Učenec prepozna napake v delovanju programa. • Učenec razume, da je dobro berljivim algoritmom enostavneje slediti, jih je lažje testirati in v zapisu najti napake (razhroščevati).
--	--

04 AKTIVNOST

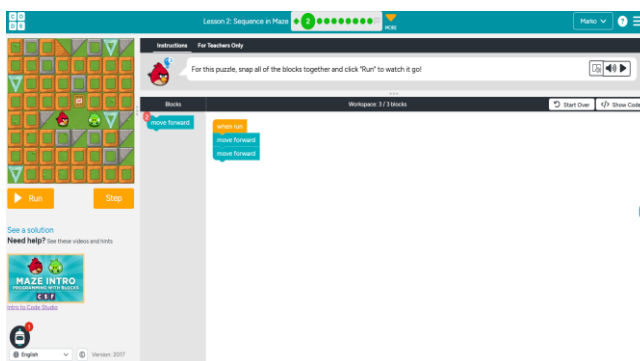
Metode poučevanja	Izkušenjsko učenje, sodelovalno učenje.
Material	• Računalnik in projektor • 5 - 10 naprav z dostopom do interneta (računalnik ali tablica)
Koraki izvedbe aktivnosti	Prvi sklop učnih aktivnosti Uro začnemo z razlago pojma algoritem in učence vprašamo, ali so se s tem pojmom že srečali. Nato jih vodimo z naslednjimi primeri in dejstvi: • Če algoritem izvaja računalnik, govorimo o računalniškem programu, če pa ga izvaja človek (npr. kuhar), gre za postopek, zapisan v obliki recepta. • Algoritme lahko zapisujemo v različnih oblikah oziroma »jezikih« – lahko so zapisani v naravnem jeziku (npr. navodila za delo) ali v programskem jeziku. • Beseda algoritem izhaja iz imena matematika Al-Hvarizmija iz 9. stoletja. • Neformalno lahko algoritem opredelimo kot zaporedje natančno določenih korakov za reševanje problema. • Poudarimo, da so algoritmi temelj delovanja digitalnih tehnologij – brez njih ne bi imeli računalnikov, telefonov ali drugih sodobnih naprav in bi sedaj to predstavitev prikazovali s pomočjo grafoskopa ali lista papirja. Nadaljujem s predstavitvijo poteka današnje ure. Za današnjo dejavnost bomo uporabili spletno orodje za programiranje Code.org, ki omogoča intuitivno učenje programiranja v vizualnem okolju. Učenci bodo spoznali blokovno programiranje, pri katerem sestavljajo program z uporabo vnaprej pripravljenih ukazov v obliki blokov, ter postopno razvijali razumevanje logike programiranja. Programsko okolje Code.org nam omogoča zapis programov v vizualni obliki ter prehod k besedilnemu programiranju. Učenci bodo spoznali pomen smiselno poimenovanih spremenljivk, uporabo že pripravljenih funkcij ter

njihovo vključevanje v lastne programe. Poleg tega bodo pri reševanju problemov iskali obstoječe rešitve, jih analizirali in prilagajali novim situacijam.

Code.org® je neprofitna organizacija, ki si prizadeva, da bi imel vsak učenec možnost učenja računalništva. Orodje nam omogoča praktično delo v znanem okolju igre, kot je Minecraft – Pustolovec, kar dodatno poveča motivacijo učencev. V tej aktivnosti se učenci seznanijo z osnovnimi koncepti programiranja (zaporedje ukazov, ponavljanje, dogodki) ter razvijajo tudi strategije za odpravljanje napak.

Potek praktičnega dela

Za izvedbo učne aktivnosti bomo uporabili orodje CS Osnove F, ki je primerno za začetnike. Učencem predhodno pripravimo namensko virtualno učilnico, v katero se vpišejo s svojim imenom. Na večpredstavnostni tabli učencem, ki še niso uporabljali orodij Code.org, predstavimo postavitev delovnega okolja, razložimo razpoložljiva orodja ter način dela. Med razlago spodbujamo vprašanja in učencem omogočimo ogled primerov reševanja začetnih lekcij. V uvodni aktivnosti učenci rešujejo naloge v igri Angry Birds, s katerimi usvojijo osnovni koncept sestavljanja ukazov z bloki. Naloge temeljijo na pravilnem zaporedju ukazov, s katerimi vodijo lik (Angry Bird) do cilja, kar učencem omogoča razvoj osnovnega algoritmičnega razmišljanja skozi igro.

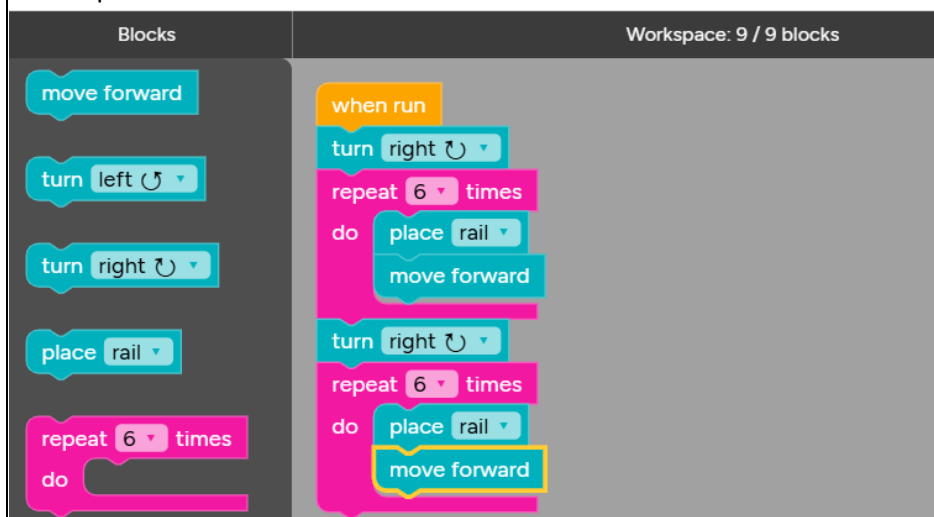


Naslednja aktivnost poteka v okolju Minecraft – Pustolovec, kjer učenci sodelujejo v igri, usmerjeni v gradnjo in raziskovanje (npr. postavljanje železnice ali ustvarjanje prehodov skozi ovire). Pri tem uporabljajo vnaprej pripravljene blokovne ukaze, s katerimi vodijo lik skozi naloge. Cilj dejavnosti je, da učenec sestavi pravilno zaporedje ukazov za uspešno rešitev naloge.

Skozi delo učenec razvija logično razmišljanje, preverja pravilnost svojih odločitev ter postopno izboljšuje svoje rešitve.



Primer pravilne rešitve:

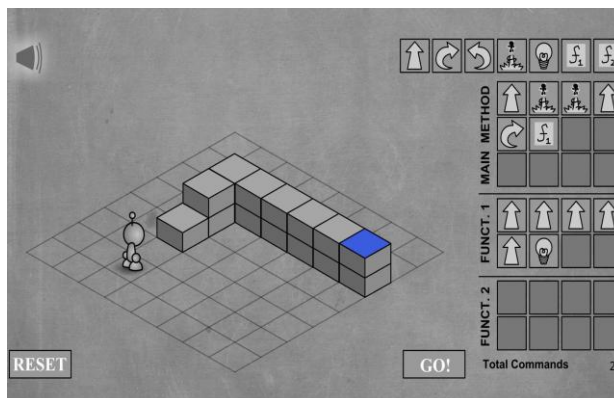


Prvi sklop aktivnosti zaključimo z evalvacijo aktivnosti, kjer učenci podajo svoja mnenja o vsebini in poteku učnih aktivnosti. Sledi odprt pogovor, v katerem skupaj razmišljamo o pojmu algoritma in pomenu pravilnega zaporedja ukazov.

2. sklop učnih aktivnosti

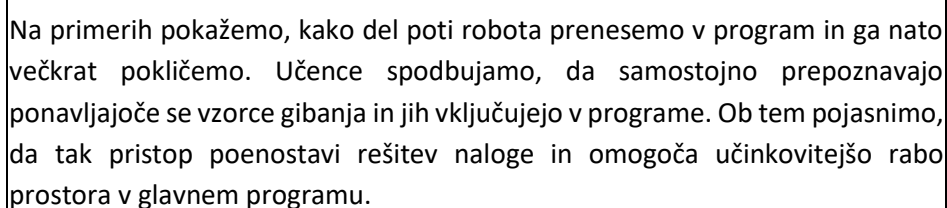
Drugi sklop učnih aktivnosti začnemo s kratkim ponavljanjem, da računalnik deluje na podlagi natančnih in zaporednih navodil. Učence spodbudimo k razmisleku v vprašanju, kaj se zgodi, če je navodilo napačno ali manjka, ter jih tako usmerimo v razumevanje pomena natančnosti pri programiranju.

Nato učencem predstavimo spletno aplikacijo **Lightbot** in jim razložimo osnovni cilj igre. Skupaj si ogledamo uporabniški vmesnik ter pojasnimo pomen posameznih ikon. Učencem predstavimo osnovne ukaze za vodenje robota (korak naprej, zavoj levo, zavoj desno) ter ukaz za aktivacijo polja. Ob tem poudarimo, da mora robot ukaze izvajati v točno določenem zaporedju, saj le tako doseže zastavljeni cilj.



V začetnih nalogah učenci rešujejo enostavne primere, pri katerih robot za dosego cilja potrebuje le krajše zaporedje ukazov. Učence spodbujamo, da pred začetkom reševanja najprej premislijo o poti robota in zaporedju korakov, šele nato začnejo z vnosom ukazov. Med delom jih usmerjamo k preizkušanju rešitev ter odpravljanju napak.

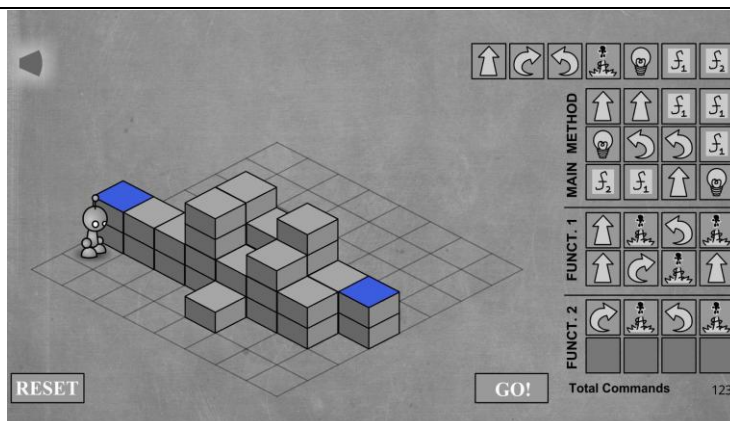
Ko učenci usvojijo osnovne ukaze, jih postopoma uvajamo v zahtevnejše naloge, kjer je zaporedje ukazov daljše in bolj kompleksno. Učence opozorimo, da je število ukazov v programskem polju omejeno, zato jim predstavimo uporabo **programov (funkcij)**. Pojasnimo, da lahko ponavljajoče se zaporedje ukazov shranijo v program in ga nato večkrat uporabijo v glavnem programu, kar prispeva k učinkovitejšemu reševanju nalog.



V nadaljevanju učenci samostojno ali v parih rešujejo naloge, pri katerih kombinirajo osnovne ukaze in programe. Med delom jih spremljamo, postavljamo usmerjevalna vprašanja ter jih spodbujamo k razmisleku o tem, zakaj določena rešitev deluje ali ne deluje.

Ob zaključku drugega sklopa učnih aktivnosti skupaj pregledamo nekaj izbranih rešitev in se pogovorimo o različnih možnih poteh do cilja. Pri tem poudarimo, da pri programiranju ne obstaja le ena pravilna rešitev, temveč več učinkovitih pristopov, ter izpostavimo pomen logičnega razmišljanja, preverjanja rešitev in vztrajnosti. Dejavnost zaključimo s spodbudo, da učenci podobno logiko uporabljajo tudi pri drugih nalogah in nadaljnjem učenju programiranja.





Uro zaključimo s spodbudo, da razmišljajo, kako lahko podobno logiko uporabijo tudi pri drugih nalogah in pri nadaljnjem učenju programiranja.

3. sklop učnih aktivnosti

3.1 Uvod

Učencem najprej predstavimo situacijo, v kateri prevzamejo vlogo programerjev, ki robotu pomagajo opraviti pomembno nalogo. Robot se odpravlja na vesoljsko misijo, pri čemer potrebuje pomoč operaterja. Učencem podamo motivacijsko zgodbo: »NASA je poslala raziskovalnega robota na misijo na Luno. Robot za uspešno delovanje potrebuje operaterja. Pomagaj mu opraviti zahtevno nalogo, na kateri se lahko zgodi marsikaj.«

Pojasnimo, da bodo skozi različne naloge postopno razvijali in nadgrajevali svoje razumevanje algoritmov – od analize problema in iskanja rešitve do njene implementacije, testiranja in vrednotenja. Ob tem izpostavimo, da gre za proces, v katerem realne situacije pretvarjamo v modele ter jih tako obravnavamo bolj pregledno, sistematično in učinkovito.

3.2 Spoznavanje orodja Pišek

Učitelj s pomočjo projekcije učencem predstavi orodje Pišek ter pojasni osnovno logiko njegove uporabe. Učence vodi skozi uporabniški vmesnik in skupaj z njimi reši nekaj uvodnih primerov, s čimer prikaže strategijo reševanja

problema: analizo besedila, prepoznavanje bistva naloge ter analiziranje modela in preverjanje omejitev.

Začnemo s problemom Zasilni pristanek. V tej fazi se osredotočimo na prvi dve nalogi, ki sta po zahtevnosti najprimernejši za uvod. Učenci ju najdejo v orodju Pišek, če sledijo naslednji poti: Učna pot - Zgodbe - Robot [49] - Misija na Luni [32] - Zasilni pristanek [3].

3.2.1 Zasilni pristanek

Zgodba učence vodi do trenutka, ko robot uspešno pristane na površini Lune, saj je moral zaradi trka z asteroidom izvesti zasilni pristanek. Njihova naloga je, da mu pomagajo najti pot do označene točke.

Stopnja 2 predstavlja uvod v uporabo orodja in osnovnih programskih gradnikov. Učenci se učijo uporabljati posamezne delčke, s katerimi se robotek premika naprej, navzgor ali navzdol. Naloga vsebuje omejitev največ 10 delčkov, kar učencem omogoča veliko svobode pri oblikovanju rešitve.

Stopnja 3 pa predstavlja prvi korak k uporabi zank. Zaradi omejitve števila delčkov so učenci primorani, da rešitev optimizirajo z uporabo zanke. V tej nalogi spoznajo gradnik "Ponavljaj x -krat, izvedi: y ", s čimer naredijo pomemben korak k bolj učinkovitemu programerskemu mišljenju.

3.3 Samostojno raziskovanje učencev.

Učence razdelimo manjše skupine, pri čemer je priporočljivo, da v vsaki skupini sodelujeta po dva učenca. Vsaka skupina potrebuje napravo z dostopom do spleta. Orodje Pišek je dostopno na podani spletni povezavi: <https://pisek.acm.si/contents/4907-4902-621190887497882908-1933429127371789301/>

Učitelj poskrbi, da vsi učenci uspešno dostopajo do orodja in se znajdejo na ustreznem mestu. Ker učenci naloge rešujejo zaporedno, učitelj sproti preverja njihov napredek in usmerja delo, da ostajajo na pravilnem izzivu. Priporočljivo je, da ima učitelj isto nalogo prikazano tudi na projektorju, kar učencem omogoča lažje sledenje poteku dela. Nabor nalog je zasnovan kot predlog, pri čemer je nujno upoštevati načelo individualizacije in diferenciacije – učitelj

lahko glede na predznanje, zmožnosti in napredek učencev prilagaja zahtevnost nalog posameznim skupinam.

3.3.1 Ovinek - Stopnja "4"

Zgodba zasilnega pristanka se nadaljuje. Tokrat se je naš robot znašel v jami, iz katere mora najti izhod. Naloga temelji na uporabi zank, saj zaradi omejitve števila razpoložljivih delčkov rešitve z linearno uporabo ukazov ni mogoče zapisati.

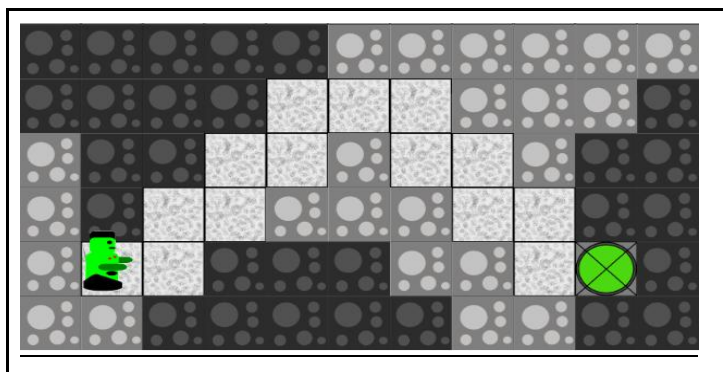
Nalogo najdemo v orodju Pišek po naslednji poti:

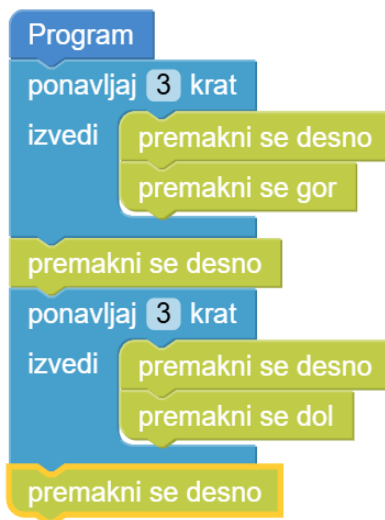
Učna pot - Zgodbe - Robot [49] - Misija na Luni [32] - Ovinek [3].

Pri tej nalogi preskočimo prvi dve stopnji zahtevnosti in se takoj lotimo stopnje 4. Učenci imajo dovoljeno uporabiti le 6 delčkov, zato morajo nujno vključiti zanko "Ponavljaj x-krat, izvedi ...". Poleg tega morajo razmišljati bolj sistematično, saj jih omejitve prisili k iskanju optimalne poti in pregledne rešitve. Naloga je odlična priložnost, da dodatno utrdimo razumevanje, zakaj so zanke pomembne ter kako omogočajo krajše, jasnejše in učinkovitejše programe.

Ker je iskanje rešitve del procesa, naloga zahteva tudi testiranje. Tako učenci že naravno vstopijo v proces načrtovanja, izvajanja in preverjanja programske rešitve, kar jim približa realen potek programiranja.

Primer rešitve:





3.3.2 Serpentina - Stopnja "3"

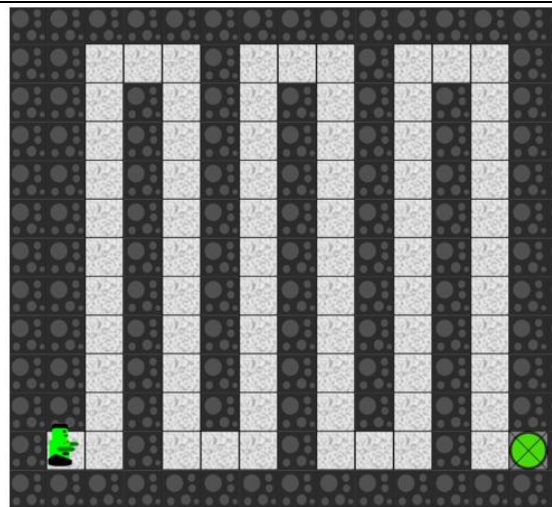
Pri tej nalogi mora robot najti izhod iz vijugaste jame. Naloga učencem omogoča spoznati koncept ugnazdenih zank, saj jih omejitev maksimalnega števila delčkov prisili k uporabi zanke znotraj zanke. Rešitev brez gnezdenja namreč ni mogoča.

Nalogo najdemo v orodju Pišek po naslednji poti:

[Učna pot - Zgodbe - Robot \[49\] - Misija na Luni \[32\] - Serpentina \[3\]](#)

Pri tej aktivnosti preskočimo prvo stopnjo zahtevnosti in se lotimo kar stopnje 3, saj ima ta v opisu naloge tudi prikazan primer rešitve. Ta sicer ni pravilna, je pa zelo uporaben kot vizualna podpora učencem. Pokaže jim, kakšno vrsto strukture pričakujemo pri reševanju serpentine. Naloga odlično razširi razumevanje zank v kontekstu gnezdenja. Zaradi kompleksnosti pa je tudi primer dobre prakse, saj bodo učenci morali aktivno preizkušati, prilagajati in "razhroščevati" svoje rešitve. Pri tej nalogi pride do izraza pristop poskus-napaka (angl. »trial and error«), ki je pomemben del »programerskega« načina razmišljanja.

Primer rešitve:



3.3.3 Paketi - Stopnja "3"

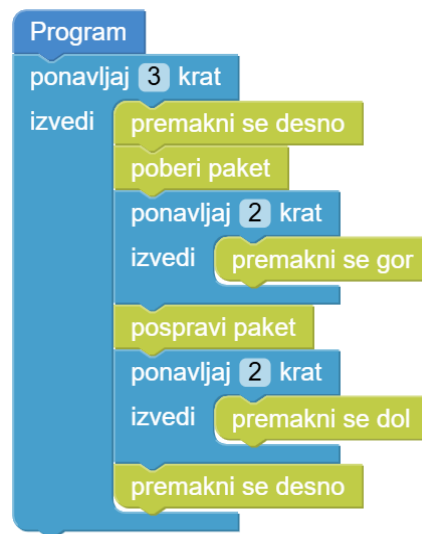
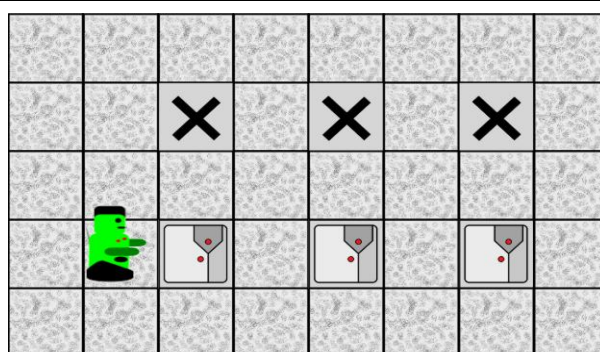
Robotu je na tej točki uspelo najti pot iz labirintov, vendar ga na površju Lune čaka nova naloga. Tam je našel veliko raztresenih orodij, ki so jih astronauti pozabili med raziskovanjem. Njegova naloga je, da pobere vse pakete in jih pravilno odloži na označena odlagališča. Naloga je primerna, ker učencem pomaga utrditi razumevanje ugnezdenih zank, saj od učencev zahteva kombiniranje gibanja robota in izvajanja dodatnih ukazov.

Nalogo najdemo v orodju Pišek po naslednji poti:

Učna pot - Zgodbe - Robot [49] - Misija na Luni [32] - Paketi [3]

Naloga je po strukturi zelo podobna prejšnji, še vedno gre za gibanje robotka skozi prostor in uporabo gnezdenih zank. Razlika je v tem, da morajo učenci poleg gibanja vključiti tudi ukaza za pobiranje in odlaganje orodja. Število delčkov je spet precej omejeno, kar jih spodbuja k iskanju optimizirane, pregledne in učinkovite rešitve.

Primer rešitve:



3.3.4 Raztreščena raketa - Stopnja "3"

Po pospravljanju orodja robot naleti na raztreščeno raketo. Ker je robot okoljevarstveno osvešččen, želi pobrati vse delčke, da ne onesnažujejo Luninega površja. Ta naloga učencem omogoča spoznavanje nove vrste zanke "for" zanke.

Nalogo najdemo v orodju Pišek po naslednji poti:

Učna pot - Zgodbe - Robot [49] - Misija na Luni [32] - Raztreščena raketa [3]

Učenci se tokrat srečajo z drugačnim tipom zanke. Ta zanka uporablja števec in pogoj za izvajanje, kar je pomembna razlika v primerjavi z zankami, ki ponavljajo določeno število korakov. V zanki določimo začetno vrednost števca i, nato pa

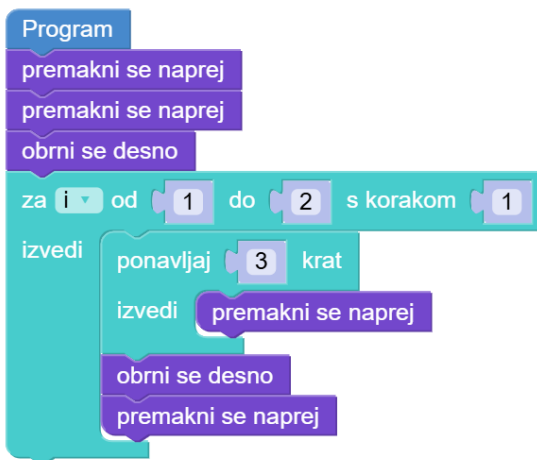
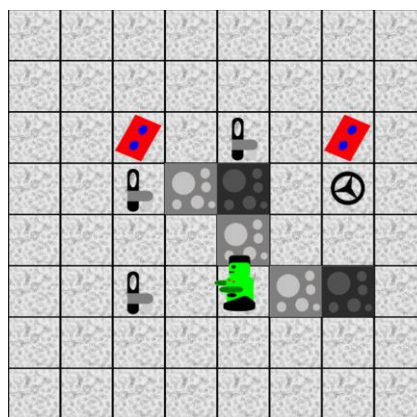
se vrednost po vsakem prehodu poveča za 1, dokler ne dosežemo meje. Primer osnovne strukture:

```
for (i = 1; i ≤ 4; i++);
```

Naloga je primer dobre prakse, saj učencem omogoča primerjavo različnih vrst zank in razumevanje, da izbor ustrezne programske strukture vpliva na učinkovitost reševanja problema. Pri reševanju učenci preizkušajo svoje rešitve, jih izboljšujejo ter odpravljajo napake, s čimer razvijajo sistematičen pristop k programiranju.

Pred začetkom reševanja je priporočljivo, da učitelj na tablo zapiše osnovno strukturo »for« zanke, jo razčleni in nazorno prikaže njen potek, da učenci lažje usvojijo njen princip delovanja.

Primer rešitve:



3.3.5 Razdejanje - Stopnja "2"

Po pospravljanju razbitin rakete robot naleti na veliko razmetanega orodja, vendar se znajde v veliki zagati. Zaradi teme in peščene nevihte ne vidi skoraj nič. V takšnih razmerah se mora zanašati na svoje senzorje. Ta naloga je odlična priložnost za vpeljavo nove vrste zanke »while« / »dokler« .

Nalogo najdemo v orodju Pišek po naslednji poti:

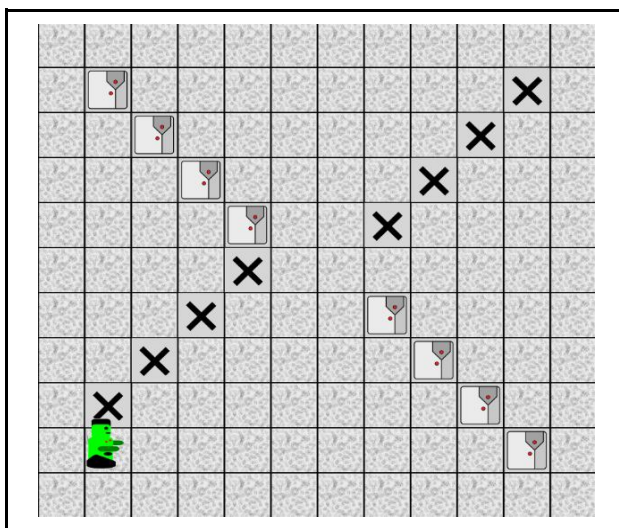
Učna pot - Zgodbe - Robot [49] - Misija na Luni [32] - Razdejanje [3]

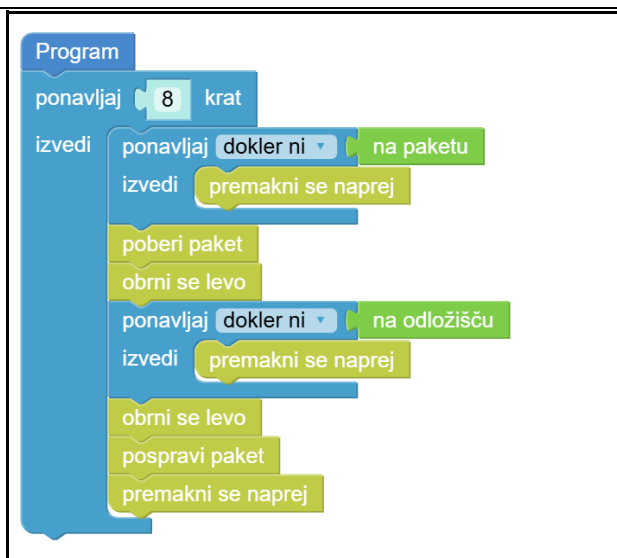
V tej nalogi učenci spoznajo zanko »while«, ki se razlikuje od prejšnjih zank, saj temelji na pogoju, ki ga moramo definirati. Zanka se lahko izvaja na dva načina:

- dokler pogoj JE izpolnjen
- dokler pogoj NI izpolnjen

Naloga od učencev zahteva uporabo celotnega nabora znanja, ki so ga pridobili v zadnjih štirih izzivih: uporabo zanke ponavljanje, gnezdenje zank, ukaza poberi in pospravi, ter sistematično načrtovanje korakov. Poleg tega morajo na rešitev zdaj pogledati še iz vidika logičnega pogoja, ki določa, kdaj se bo zanka ustavila. Z uvajanjem zanke "dokler" učenci svoje algoritmično razmišljanje dodatno nadgradijo ter razumejo, kako se program lahko prilagaja okolju in pogojem kar je izjemno pomembno v realnih »programerskih« situacijah.

Primer rešitve:





3.3.6 Preučevanje - Stopnja "3"

Med raziskovanjem Luninega površja robot naleti tudi na mesta, označena z velikim "X", ki zahtevajo posebno pozornost. Ker je jama, v kateri se nahaja, ozka in vijugasta, potrebuje dodatno pomoč pri premikanju in odločanju. Ta naloga je idealna za uvajanje koncepta pogojnih stavkov "če" / "if".

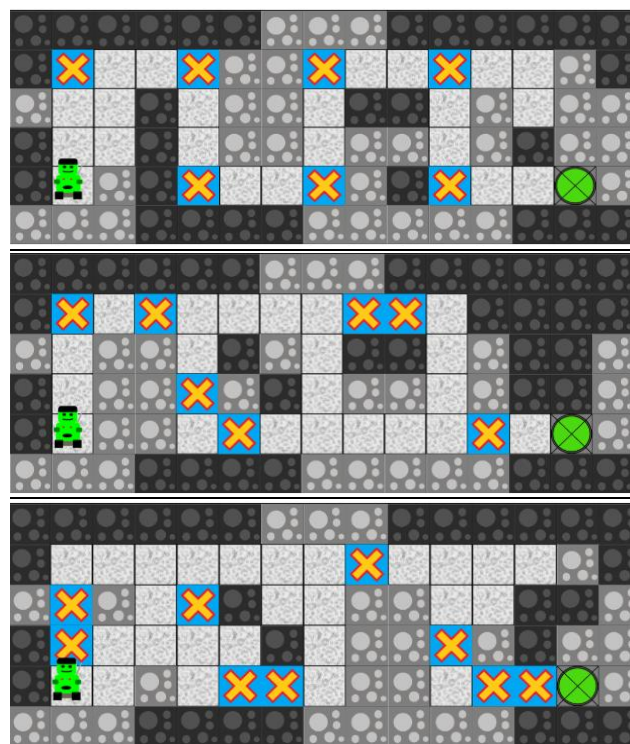
Nalogo najdemo v orodju Pišek po naslednji poti:
 Učna pot - Zgodbe - Robot [49] - Misija na Luni [32] - Preučevanje [3]

Tako kot prejšnja je tudi ta naloga primer dobre prakse, saj združuje vse doslej osvojene elemente: uporabo zank, gnezdenje zank, ukaze za dejanja (poberi, pospravi, premakni se ...), ter zdaj še pogojni stavek "if". Učenci se naučijo, da program lahko sprejema odločitve glede na razmere v okolju:

- če je nekaj res, potem izvedi določeno dejanje.

Ta naloga učence spodbuja k bolj prilagodljivemu razmišljanju, saj morajo poleg zaporedja korakov upoštevati tudi pogoje, ki vplivajo na nadaljnje izvajanje programa.

Primer rešitve:



	3.4 Zaključek, refleksija in evalvacija Ob koncu zadnjega sklopa učnih aktivnosti učenci zaključijo svojo vesoljsko misijo in se simbolno vrnejo iz vloge programerjev, ki so robotu pomagali pri raziskovanju Luninega površja. Sledi vodena refleksija, v kateri skupaj pregledamo potek aktivnosti ter izpostavimo ključne koncepte, ki so jih pri tem usvojili: različne vrste zank (ponavljalj, for/za, while/dokler), pogojne stavke (if/če), gnezdenje zank ter pomen sistematičnega pristopa k reševanju problemov. Učence usmerimo k razumevanju, da programiranje zahteva natančnost, logično razmišljanje ter vztrajnost pri preizkušanju in izboljševanju rešitev. Aktivnost zaključimo s kratkim povzetkom naučenega, ob tem pa učitelj izpostavi primere uspešnih rešitev in spodbuja občutek dosežka, sodelovanja ter ustvarjalnosti učencev. V refleksiji učencem zastavimo nekaj odprtih vprašanj, ki jih spodbudijo k razmisleku o lastnem učenju in procesu reševanja nalog. Vprašanja obravnavamo ustno ali v parih (npr. kaj jim je bilo najbolj zanimivo, katera naloga jim je predstavljala največji izziv, kako so prišli do rešitve in kako razumejo uporabo posameznih programskih struktur). S tem učence usmerimo v poglobljeno razmišljanje o algoritmičnih, zaporedju ukazov, zankah in pogojnih stavkih ter njihovi uporabnosti v praksi. Dejavnost zaključimo z izpolnjevanjem kratkega evalvacijskega vprašalnika, s katerim preverimo razumevanje snovi ter pridobimo vpogled v njihova doživetja in mnenja o učnem procesu.
Video in slikovno gradivo	/

05 EVALVACIJA, REFLEKSIJA

Evalvacija	1. Opišite situacije, na podlagi katerih lahko sklepate, da so se v procesu učenja začeli realizirati zastavljeni cilji. Na realizacijo ciljev sklepamo na podlagi aktivnega sodelovanja učencev in njihove uspešnosti pri reševanju nalog. Učenci so že v začetnih učnih aktivnostih pokazali razumevanje zaporedja ukazov, kar se je odražalo v uspešnem reševanju nalog v Angry Birds in Minecraftu. Pri nadaljnjih nalogah so prepoznavali ponavljajoče se vzorce in jih vključili v programe, v zahtevnejših dejavnostih v Pišku pa so samostojno prepoznavali ustrezne strukture (zanke, pogoje), popravljali napake in optimizirali svoje rešitve. Pri tem so pogosto
-------------------	---

	razlagali svoje postopke, primerjali rešitve z drugimi ter povezovali novo znanje s predhodnimi izkušnjami. 2. Pri katerih elementih dejavnosti so bili učenci uspešni, delno uspešni ali neuspešni? Učenci so bili uspešni predvsem pri razumevanju zaporedja ukazov, prepoznavanju vzorcev in uporabi osnovnih programskih struktur, saj so večino enostavnih nalog reševali samostojno. Delno uspešni so bili pri zahtevnejših dejavnostih, zlasti pri optimizaciji rešitev, gnezdenju zank in uporabi pogojnih stavkov, kjer so nekateri potrebovali dodatno usmerjanje. Največ izzivov so predstavljale naloge z omejenim številom ukazov, saj so zahtevale bolj učinkovito in premišljeno rešitev. Neuspešnih učencev ni bilo, saj so ob težavah poiskali pomoč in z dodatnim razmislekom ali sodelovanjem našli ustrezne rešitve. 3. Katere cilje so učenci dosegli in katerih mogoče ne, zakaj? Učenci so dosegli cilje, povezane z razumevanjem zaporedja ukazov, prepoznavanjem vzorcev, uporabo zank ter samostojnim reševanjem nalog. Dobro so razumeli tudi prenos stvarnega problema v računalniški problem ter pomen testiranja in odpravljanja napak (razhroščevanja), saj so vztrajno preverjali delovanje kode in popravljali napake. Delno pa so bili doseženi cilji, povezani z optimizacijo rešitev ter kompleksnejšim kombiniranjem krmilnih struktur, saj so nekateri učenci pri gnezdenju zank in uporabi pogojnih stavkov potrebovali več časa, dodatna vprašanja ali pomoč. Kljub temu so postopoma izboljševali svoje rešitve, kar kaže na napredek v razumevanju. 4. Kako/na kakšen način ste preverjali doseganje ciljev? Doseganje ciljev smo preverjali sproti z opazovanjem dela učencev med učnimi aktivnostmi, pogovorom in postavljanjem usmerjevalnih vprašanj. Spremljali smo njihovo sposobnost razumevanja zaporedja ukazov, uporabo programskih struktur ter način razmišljanja pri reševanju nalog. Učenci so razlagali svoje rešitve, utemeljevali izbire ter primerjali pristope z vrstniki. Posebno pozornost smo namenili preverjanju, ali znajo prepoznati napake, jih popraviti in izboljšati rešitve. Doseganje ciljev smo preverjali tudi z uspešnostjo rešenih nalog ter z evalvacijskim vprašalnikom, ki je omogočil vpogled v razumevanje in njihov odnos do učenja.
Refleksija z učečimi se	1. Kaj je bilo otrokom/učencem všeč in zakaj? “Najbolj mi je bilo všeč delo v parih, ker smo se malo boljše spoznali.” “ Minecraft”

	“ Zadnja delavnica, ker smo tekmovali” “ Vse zato ker je bilo zanimivo” “Najbolj všeč mi je bila možnost, da smo lahko sami kodirali in se učili o programiranju” “ Ko sem sam igral, ker mi je bilo zanimivo.” “ Ko smo igrali minecraft” “ Najbolj light bot.” “ Zadnja delavnica z tekmovanjem, ker smo zmagale.” “ Najbolj mi je bila 1 aletunost.” “ Tisti robotek z lučko, ker mi je bilo zanimivo.” “ Ko smo igrali / programirali.” “ Tekmovanje, saj sem tekmovalna oseba.” “ Ki smo programirali.” “ Use ta zadnja.” 2. Kaj so otroci/učenci po njihovem mnenju spoznali, ugotovili? “ Da obstaja program za tekmovanje.” “Kako se programira” “ Kaj so bloki in kako delujejo.” “ Algoritmi nisem znala kaj pomeni zdaj pa ja.” “ kode” “ Da je kodiranje lahko zabavno.” “ robotke “ “ Kako deluje programiranje in njegov postopek.” “ da je kodiranje lahko zanimivo” “ vse možno.” “ Da lahko samo daš zanko in ne rabiš pisati več.” 3. S kom in kako so sodelovali, zakaj? “ Sodelovala sem z *** ker moje sošolke ni bilo.” “ **** dobro.” “ *** je bila moja so-tekmovalka.” “****” “ Z **** . ker je moj prijatelj” “ Z mojim sošolcem, ker sva sedela zraven.” “ Z sošolcem” “ Z ****, mila je moja sotekmovalka. Sedela je zraven mene.” “ Z sošolcem smo mogli sodelovati.” “ **** in ****”
--	--

	“ Z **** in **, ker so nas tako dali v skupine.” “ Z ****, ker je pameten.” “ Z ****. Smo pač rabili” “ Sam nakoncu s dvema ki nista nič delala”. 4. Kako so se počutili, kaj so doživljali? “ Stresno in lepo” “ Pametnejši od ****. “ “Dobro.” “ Zelo dobro.” “ Počutil sem se vredu.” “ OK.” “ Zelo mi je bilo zabavno.” “ Zanimivo” “ SUPER “ “ Odlično, saj smo se veliko zabavali, igrali , smejali...” 5. Kaj bi učeči se spremenili? “Nič.” “ Več časa bi vzela za določeno delavnico saj nam ga je na koncu zmanjkalo.” “ Da bi delali zunaj.” “ Nič, vse je vredu. Mogoče, da kdo hoče sam delat lahko sam.” “Igranje z pravimi roboti namesto tistimi na ekranu.” 6. Pobude in predlogi otrok/učencev v procesu izvedbe in evalvacije: “ Naj ostane tako zanimivo.” “ Več tekmovalj” “ Ukvarjanje z robotki in skrivalnice po šoli :) “ “ Igranje kakšne igrice in ugotavljanje kode. “ “ Kodiranje za robote.” “ Več interaktivnih dejavnosti na prosto.” “ Bi se sprehajali” “ Več gibanja.” “ Nevem, določite vi.”
Profesionalna refleksija načrtovanja in izvedbe	1. Na podlagi izvedbe ocenite ustreznost načrtovanja in izvedbe procesa učenja in poučevanja za otroke v skupini/učence v razredu (kaj ocenjujete kot uspešno načrtovanje in kaj bi spremenili/dopolnili)?

Načrtovanje in izvedbo ocenjujemo kot ustrezno, saj je temeljila na uporabi interaktivnih metod (iger), ki spodbujajo motivacijo, aktivno sodelovanje in logično razmišljanje. Učenci so imeli možnost samostojno preizkušati zaporedja ukazov, uporabljati programe ter prilagajati kodo za reševanje novih problemov, kar je pomembno prispevalo k doseganju ciljev. Postopno uvajanje od enostavnih nalog do zahtevnejših kombinacij je omogočilo nadgradnjo znanja in prilagajanje različnim sposobnostim učencev. Kot uspešno se je izkazala tudi prilagoditev razlage glede na predznanje in poznavanje okolij (npr. Minecraft). Med izboljšavami izpostavljamo potrebo po več usmerjevalnih vprašanjih za manj uspešne učence, dodatnih demonstracijah optimalnih rešitev ter boljši časovni razporeditvi, pri čemer bi bilo smiselno zmanjšati število nalog in posameznim nameniti več časa za poglobljeno razumevanje in refleksijo.

2. Kako ste prilagodili aktivnost glede na starost, razvojne značilnosti, (pred)znanje, kulturne in jezikovne značilnosti in druge značilnosti otrok?

Učne aktivnosti smo prilagodili starosti, predznanju in razvojnim značilnostim učencev z postopnim prehodom od uvodnega spoznavanja pojma algoritma do praktičnega reševanja nalog v znanih digitalnih okoljih. Izkazalo se je, da učenci osnove dela v teh okoljih že obvladajo, zato bi lahko začetne razlage delno skrajšali. Aktivnosti smo nadgradili z uvajanjem zahtevnejših nalog, pri katerih so učenci uporabljali programe (funkcije) in kombinirali različne ukaze. Diferenciacijo smo zagotavljali tako, da so bolj sposobni učenci samostojno raziskovali in reševali zahtevnejše izzive, manj sposobnim pa smo nudili dodatno podporo in usmerjevalna vprašanja. Naloge so bile podane jasno in vizualno, spodbujali smo delo v parih, da so si učenci medsebojno pomagali in izmenjevali ideje. Učence z manjšim predznanjem ali jezikovnimi ovirami smo smiselno vključili v skupine, kjer so dobili dodatno podporo, pri izbiri nalog pa smo upoštevali, da so dovolj konkretne in vizualne, da omogočajo razumevanje ne glede na jezikovni nivo. Tempo dela smo prilagodili tako, da so imeli vsi učenci dovolj časa za razmislek, sodelovanje in razpravo.

3. Zakaj je izvedba dejavnosti lahko primer dobre prakse?

Izvedba učnih aktivnosti je primer dobre prakse, ker temelji na aktivnem, izkustvenem in problemskem učenju v učencem znanem okolju, kar pozitivno vpliva na motivacijo in uspešnost vseh učencev. Učenci so imeli možnost samostojno preizkušati zaporedja ukazov, reševati naloge in postopno nadgrajevati znanje, od osnovnih ukazov do zahtevnejših kombinacij z uporabo programskih struktur. Aktivnosti so bile prilagojene različnim sposobnostim, spodbujale so samostojnost, logično razmišljanje in

	sodelovanje, hkrati pa omogočale, da učenci aktivno preverjajo in izboljšujejo svoje rešitve. Učili so se skozi izkušnjo, raziskovanje in razpravo, pri čemer so razvijali računalniško mišljenje in medsebojno komunikacijo. Naloge so bile diferencirane, jasno povezane z učnimi cilji, učitelj pa je sproti spremljal razumevanje in nudil ustrezno podporo, kar skupaj predstavlja kakovosten model sodobnega poučevanja. 4. Kje ste imeli največ težav in katere predloge izboljšav predlagate? Največ težav smo imeli pri ustreznem prilagajanju zahtevnosti nalog in časovni razporeditvi aktivnosti. Uvodne naloge so bile za večino učencev nekoliko premalo zahtevne glede na njihovo predznanje, medtem ko so se pri zahtevnejših nalogah nekateri učenci zatikali predvsem pri uporabi programskih struktur, prepoznavanju ponavljajočih se vzorcev in optimizaciji rešitev. Dodatni izziv je predstavljalo tudi časovno načrtovanje, saj so nekatere skupine potrebovale več časa za razumevanje in reševanje posameznih nalog. V prihodnje bi naloge postavili na nekoliko višjo začetno raven, dodali več vodilnih vprašanj in kratke demonstracije optimalnih rešitev, uvedli vmesne postanke za razjasnitev ključnih konceptov ter zmanjšali število nalog, da bi lahko posameznim dejavnostim namenili več časa za poglobljeno razumevanje, razpravo in refleksijo.
Drugi komentarji	/

06 KURIKULUM

Medpredmetno povezovanje	
Povezanost s skupnimi cilji učnih načrtov (označiti in kratko opredeliti)	1. Trajnostni razvoj Učenci se učijo učinkovitega reševanja problemov in optimizacije postopkov, kar spodbuja premišljeno in odgovorno rabo virov. 2. Digitalne kompetence Učenci razvijajo algoritmično razmišljanje, uporabljajo digitalna orodja za programiranje ter razumejo delovanje programov in odpravljanje napak. 3. Zdravje in dobrobit Sodelovalno delo in igrifikacija povečujejo motivacijo, zmanjšujejo stres in krepijo pozitivno učno klimo. 4. Podjetnost Učenci razvijajo samostojnost, ustvarjalnost, sodelovanje in iskanje učinkovitih rešitev pri reševanju problemskih nalog.